Agile Development Process and Devops

NAME; MADHU PRIYA V

REG NO ;23MIS0623

INVENTORY MANAGEMENT SYSTEM

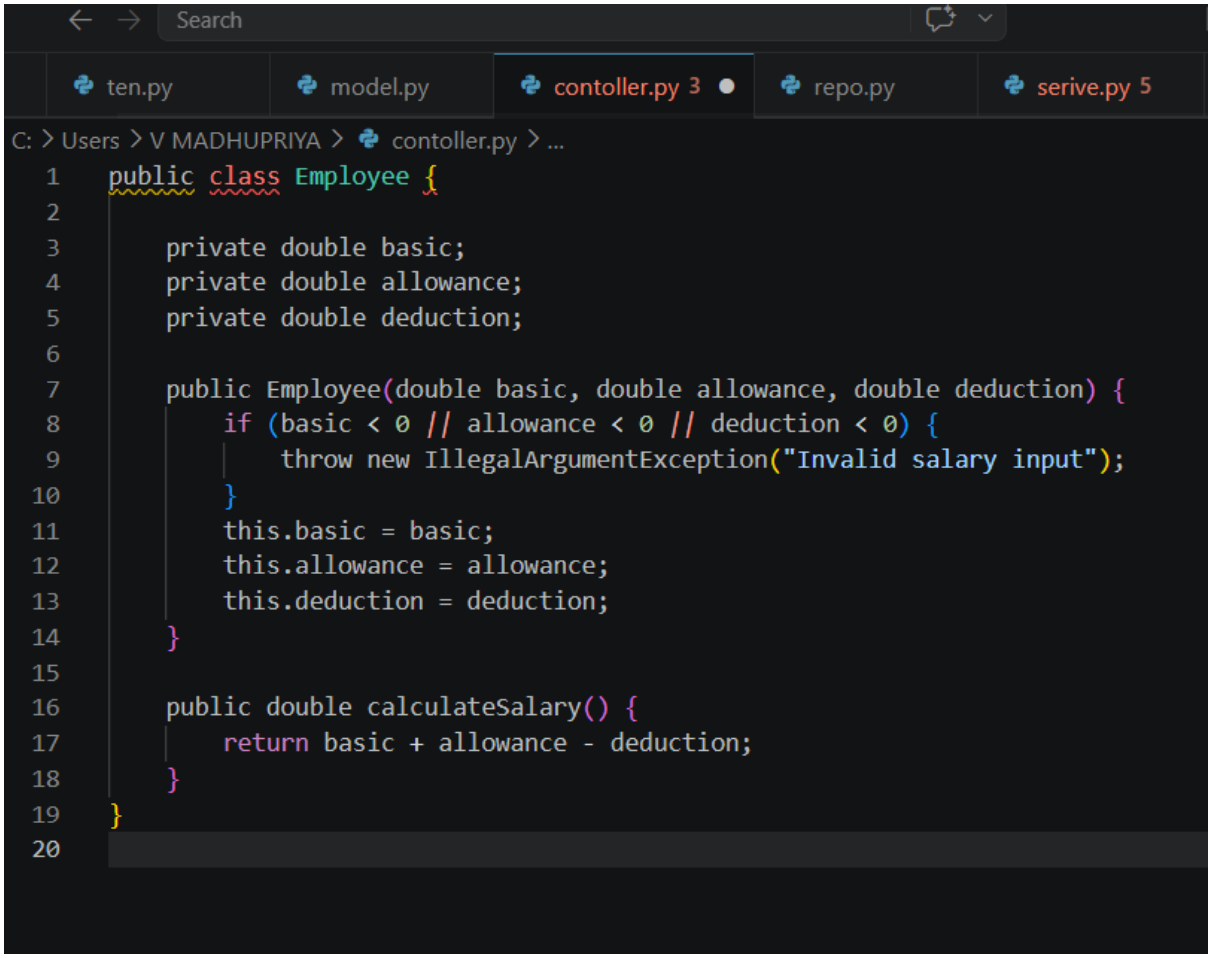
Problem Statement:
Ensure inventory levels are updated sccurately after stock changes.
Tasks:

TASK;

1,Develop stock update logic

```
package com.example.inventory.service;
import com.example.inventory.exception.InsufficientStockException;
import com.example.inventory.model.InventoryItem;
import com.example.inventory.repository.InventoryRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
@Service @RequiredArgsConstructor
public class InventoryServiceImpl implements InventoryService {
private final InventoryRepository repo;
public InventoryItem createItem(InventoryItem i) { return repo.save(i); }
public List<InventoryItem> getAll() { return repo.findAll(); }
public InventoryItem getById(Long id) {
return repo.findById(id).orElseThrow(() -> new RuntimeException("Not found: "+id));
}
public InventoryItem updateItem(Long id, InventoryItem u) {
InventoryItem ex = getById(id);
ex.setProductName(u.getProductName()); ex.setQuantity(u.getQuantity());
```

```
ex.setPrice(u.getPrice()); return repo.save(ex);
}
public void deleteItem(Long id) { repo.delete(getById(id)); }
@Transactional
public InventoryItem addStock(Long id, int qty) {
if (qty <= 0) throw new IllegalArgumentException("Qty must be > 0");
repo.addStock(id, qty); // ← ADD STOCK logic
return getById(id);
}
@Transactional
public InventoryItem reduceStock(Long id, int qty) {
InventoryItem item = getById(id);
if (item.getQuantity() < qty)
throw new InsufficientStockException( // ← VALIDATION
"Insufficient stock. Available: " + item.getQuantity());
repo.reduceStock(id, qty); // ← REDUCE STOCK logic
return getById(id);
}
public List<InventoryItem> getLowStock() {
return repo.findAll().stream()
.filter(i -> i.getQuantity() < i.getReorderLevel()).toList();
}
}
```



•2,Write JUnit tests for:

Inventory Management System — Project Report
Page 8IMS Report | CSE Dept
9. JUNIT TEST CASES
InventoryServiceTest.java — Unit Tests
Java

```
package com.example.inventory;
import com.example.inventory.exception.InsufficientStockException;
import com.example.inventory.model.InventoryItem;
import com.example.inventory.repository.InventoryRepository;
import com.example.inventory.service.InventoryServiceImpl;
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.*;
```

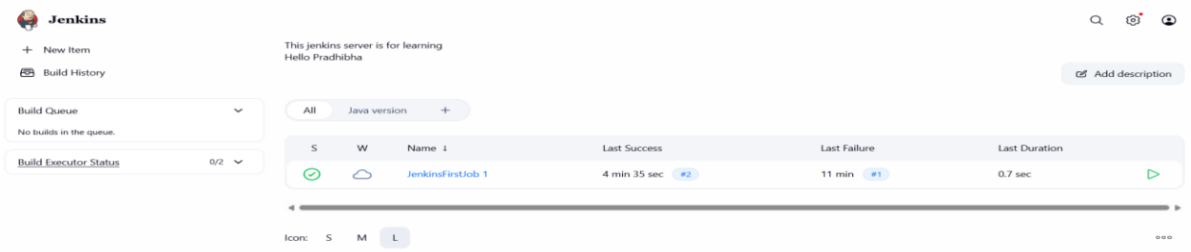
```
import org.mockito.junit.jupiter.MockitoExtension;
import java.util.Optional;
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;
@ExtendWith(MockitoExtension.class)
class InventoryServiceTest {
@Mock InventoryRepository repo;
@InjectMocks InventoryServiceImpl service;
private InventoryItem item;
@BeforeEach void setUp() {
item=new InventoryItem(); item.setId(1L);
item.setProductName("Widget A"); item.setQuantity(100);
item.setReorderLevel(10);
}
@Test void testCreateItem_Success() {
when(repo.save(item)).thenReturn(item);
assertEquals("Widget A",service.createItem(item).getProductName());
}
@Test void testGetById_Success() {
when(repo.findById(1L)).thenReturn(Optional.of(item));
assertEquals(100,service.getById(1L).getQuantity());
}
@Test void testAddStock_Success() {
when(repo.findById(1L)).thenReturn(Optional.of(item));
when(repo.addStock(1L,50)).thenReturn(1);
assertNotNull(service.addStock(1L,50)); verify(repo).addStock(1L,50);
}
@Test void testReduceStock_Success() {
when(repo.findById(1L)).thenReturn(Optional.of(item));
when(repo.reduceStock(1L,30)).thenReturn(1);
assertNotNull(service.reduceStock(1L,30)); verify(repo).reduceStock(1L,30);
}
@Test void testReduceStock_InsufficientStock() {
when(repo.findById(1L)).thenReturn(Optional.of(item));
assertThrows(InsufficientStockException.class,()->service.reduceStock(1L,999));
}
@Test void testUpdateItem_Success() {
```

```
when(repo.findById(1L)).thenReturn(Optional.of(item));
when(repo.save(any())).thenReturn(item);
assertNotNull(service.updateItem(1L,item));
}
@Test void testDeleteItem_Success() {
when(repo.findById(1L)).thenReturn(Optional.of(item));
doNothing().when(repo).delete(item);
assertDoesNotThrow(()->service.deleteItem(1L));
}
}
```

```
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.138 s
[INFO] Finished at: 2026-03-02T18:40:02+05:30
```

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.payroll</groupId>
    <artifactId>payroll-system</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <!-- JUnit -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
```

```
        <artifactId>junit-jupiter</artifactId>
        <version>5.9.0</version>
        <scope>test</scope>
      </dependency>
    </dependencies>

    <build>
      <plugins>
        <!-- Surefire plugin for running tests -->
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.0.0-M5</version>
        </plugin>
      </plugins>
    </build>
</project>
```



3, CI/CD pipeline for automated testing

YAML;

```
name: Inventory CI/CD
on:
push: { branches: [main] }
pull_request: { branches: [main] }
jobs:
```

build-test-deploy:
runs-on: ubuntu-latest
services:
mysql:
image: mysql:8.0
env: { MYSQL_ROOT_PASSWORD: root, MYSQL_DATABASE: inventorydb }
ports: ['3306:3306']
options: --health-cmd="mysqladmin ping" --health-interval=10s
steps:
- uses: actions/checkout@v4
- name: Set up JDK 17
uses: actions/setup-java@v4
with: { java-version: '17', distribution: 'temurin' }
- name: Build & Run JUnit Tests
run: mvn clean verify --no-transfer-progress
- name: Build Docker Image
run: docker build -t ${{ secrets.DOCKER_USERNAME }}/inventory-service:${{ github.sha }} .
- name: Push to Docker Hub
run: |
echo "${{ secrets.DOCKER_PASSWORD }}" | docker login -u "${{ secrets.DOCKER_USERNAME }}" --passwor
docker push ${{ secrets.DOCKER_USERNAME }}/inventory-service:${{ github.sha }}
- name: Deploy to Kubernetes
run: kubectl set image deployment/inventory-service inventory-service=${{ secrets.DOCKER_USERNAME }}

4,\DOCKER FILE

11. DOCKER SETUP
Dockerfile & docker-compose.yml
Java

```
# Dockerfile — Multi-stage build
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q
COPY src ./src
RUN mvn clean package -DskipTests -q
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/inventory-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]


HEALTHCHECK --interval=30s CMD wget -q --spider http://localhost:8080/actuator/health || exit 1
```

YAML

```
# docker-compose.yml
version: '3.8'
services:
mysql:
image: mysql:8.0
environment: { MYSQL_ROOT_PASSWORD: root, MYSQL_DATABASE: inventorydb }
ports: ["3306:3306"]
volumes: [mysql-data:/var/lib/mysql]
healthcheck:
test: ["CMD","mysqladmin","ping","-h","localhost"]
interval: 10s
inventory-service:
build: .
ports: ["8080:8080"]
environment:
SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/inventorydb
SPRING_DATASOURCE_USERNAME: root
SPRING_DATASOURCE_PASSWORD: root
depends_on: { mysql: { condition: service_healthy } }
volumes:
mysql-data:
```

Deploy on Kubernetes

12. KUBERNETES SETUP

Deployment & Service Manifests

YAML

```
# k8s/deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
name: inventory-service
spec:
replicas: 2
selector: { matchLabels: { app: inventory-service } }
template:
metadata: { labels: { app: inventory-service } }
spec:
containers:
- name: inventory-service
image: yourdockerhub/inventory-service:latest
ports: [{ containerPort: 8080 }]
env:
- { name: SPRING_DATASOURCE_URL, value: jdbc:mysql://mysql-service:3306/inventorydb }
readinessProbe:
httpGet: { path: /actuator/health, port: 8080 }
initialDelaySeconds: 30
resources:
requests: { memory: "256Mi", cpu: "250m" }
limits: { memory: "512Mi", cpu: "500m" }
```

YAML

```
# k8s/service.yml
apiVersion: v1
kind: Service
metadata:
name: inventory-service
spec:
selector: { app: inventory-service }
ports: [{ port: 80, targetPort: 8080 }]
type: LoadBalancer
# Deploy commands:
# kubectl apply -f k8s/
# kubectl get pods
# kubectl get svc inventory-service
```